

Lab Assignment 05
Title - Data Analytics II

Aim:

Implement logistic regression and calculate the confusion matrix to find TP, FP, TN, FN, Accuracy, Error rate, Precision, Recall on the Social_Network_Ads.csv dataset.

Problem Statement:

Data Analytics II

1. Implement logistic regression using Python/R to perform classification on Social_Network_Ads.csv dataset.
2. Compute Confusion matrix to find TP, FP, TN, FN, Accuracy, Error rate, Precision, Recall on the given dataset.

Objective:

Students should be able to data analysis using logistic regression using Python for any open source dataset.

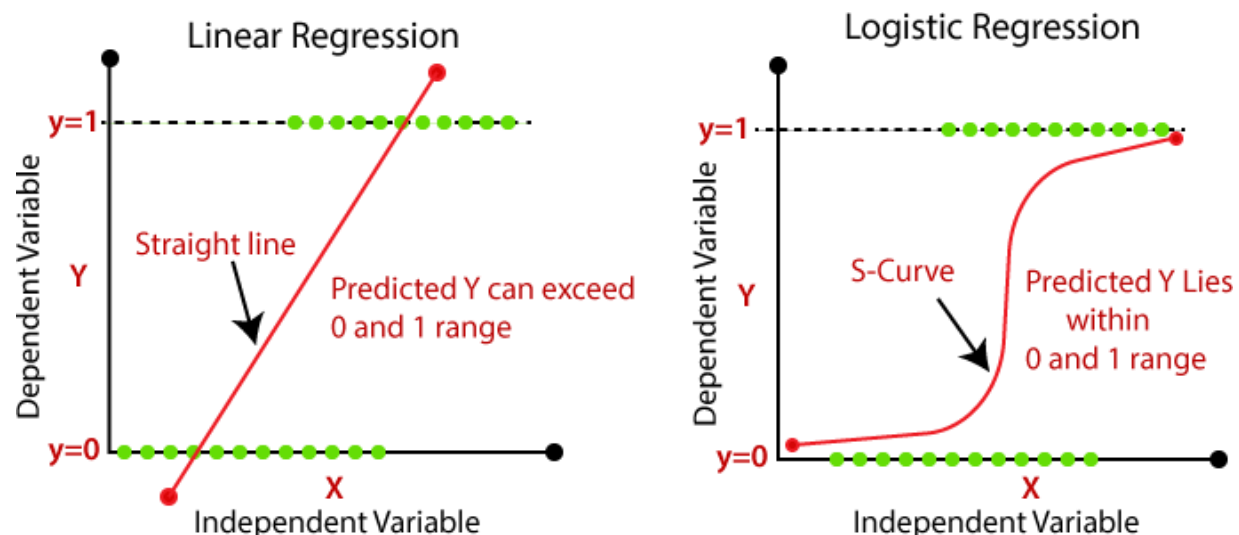
Prerequisite:

1. Basic of Python Programming
2. Concept of Regression.
3. Confusion Matrix

Logistic regression is a statistical algorithm which analyze the relationship between two data factors.

Linear Regression vs Logistic Regression

Linear Regression and Logistic Regression are the two famous Machine Learning Algorithms which come under supervised learning technique. Since both the algorithms are of supervised in nature hence these algorithms use labeled dataset to make the predictions. But the main difference between them is how they are being used. The Linear Regression is used for solving Regression problems whereas Logistic Regression is used for solving the Classification problems. The description of both the algorithms is given below along with difference table.



Logistic Regression:

- **Logistic regression is one of the most popular Machine learning algorithm that comes under Supervised Learning techniques.**
- **It can be used for Classification as well as for Regression problems, but mainly used for Classification problems.**
- **Logistic regression is used to predict the categorical dependent variable with the help of independent variables.**
- **The output of Logistic Regression problem can be only between 0 and 1.**
- **Logistic regression can be used where the probabilities between two classes is required. Such as whether it will rain today or not, either 0 or 1, true or false etc.**

- **Logistic regression is based on the concept of Maximum Likelihood estimation. According to this estimation, the observed data should be most probable.**
- **In logistic regression, we pass the weighted sum of inputs through an activation function that can map values in between 0 and 1. Such activation function is known as sigmoid function and the curve obtained is called as sigmoid curve or S-curve. Consider the below image:**
- **The equation for logistic regression is:**

$$\log \left[\frac{y}{1-y} \right] = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \dots + b_nx_n$$

Logistic Function – Sigmoid Function

- The sigmoid function is a mathematical function used to map the predicted values to probabilities.
- It maps any real value into another value within a range of 0 and 1. The value of the logistic regression must be between 0 and 1, which cannot go beyond this limit, so it forms a curve like the “S” form.
- The S-form curve is called the Sigmoid function or the logistic function.
- In logistic regression, we use the concept of the threshold value, which defines the probability of either 0 or 1. Such as values above the threshold value tends to 1, and a value below the threshold values tends to 0.

Write down formula for sigmoid function

Types of Logistic Regression

On the basis of the categories, Logistic Regression can be classified into three types:

1. **Binomial:** In binomial Logistic regression, there can be only two possible types of the dependent variables, such as 0 or 1, Pass or Fail, etc.
2. **Multinomial:** In multinomial Logistic regression, there can be 3 or more possible unordered types of the dependent variable, such as “cat”, “dogs”, or “sheep”
3. **Ordinal:** In ordinal Logistic regression, there can be 3 or more possible ordered types of dependent variables, such as “low”, “Medium”, or “High”.

Assumptions of Logistic Regression

We will explore the assumptions of logistic regression as understanding these assumptions is important to ensure that we are using appropriate application of the model. The assumption

include:

1. Independent observations: Each observation is independent of the other. meaning there is no correlation between any input variables.
2. Binary dependent variables: It takes the assumption that the dependent variable must be binary or dichotomous, meaning it can take only two values. For more than two categories SoftMax functions are used.
3. Linearity relationship between independent variables and log odds: The relationship between the independent variables and the log odds of the dependent variable should be linear.
4. No outliers: There should be no outliers in the dataset.
5. Large sample size: The sample size is sufficiently large

Working of Logistic Regression:

Logistic regression works in the following steps:

1. **Prepare the data:** The data should be in a format where each row represents a single observation and each column represents a different variable. The target variable (the variable you want to predict) should be binary (yes/no, true/false, 0/1).
2. **Train the model:** We teach the model by showing it the training data. This involves finding the values of the model parameters that minimize the error in the training data.
3. **Evaluate the model:** The model is evaluated on the held-out test data to assess its performance on unseen data.
4. **Use the model to make predictions:** After the model has been trained and assessed, it can be used to forecast outcomes on new data.

Confusion Matrix:

In machine learning, Classification is used to split data into categories. But after cleaning and preprocessing the data and training our model, how do we know if our classification model performs well?

Classification Models have multiple categorical outputs. Most error measures will calculate the total error in our model, but we cannot find individual instances of errors in our model. The model might misclassify some categories more than others, but we cannot see this using a standard accuracy measure.

A confusion matrix presents a table layout of the different outcomes of the prediction and results of a classification problem and helps visualize its outcomes.

		Actual	
		Positive	Negative
Predicted	Positive	True Positive	False Positive
	Negative	False Negative	True Negative

Figure 2: Confusion Matrix

- **True Positive:** The number of times our actual positive values are equal to the predicted positive. You predicted a positive value, and it is correct.
- **False Positive:** The number of times our model wrongly predicts negative values as positives. You predicted a negative value, and it is actually positive.
- **True Negative:** The number of times our actual negative values are equal to predicted negative values. You predicted a negative value, and it is actually negative.
- **False Negative:** The number of times our model wrongly predicts negative values as positives. You predicted a negative value, and it is actually positive.

To find how accurate our model is, we use the following metrics:

- **Accuracy:** The accuracy is used to find the portion of correctly classified values. It tells us how often our classifier is right. It is the sum of all true values divided by total values.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

- **Precision:** Precision is used to calculate the model's ability to classify positive values correctly. It is the true positives divided by the total number of predicted positive values.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall:** It is used to calculate the model's ability to predict positive values. "How often does the model predict the correct positive values?". It is the true positives divided by the total number of actual positive values.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-Score:** It is the harmonic mean of Recall and Precision. It is useful when you need to take both Precision and Recall into account.

$$\text{F1-Score} = \frac{2 * \text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

Algorithm:

1. Import python libraries

```
import pandas as pd
```

```
import numpy as np
```

2. Read the dataset

```
dataset=pd.read_csv(r"C:\Users\Student\Downloads\archive\Social_Network_Ads.csv")
```

3. Split the dataset into testing and training

```
from sklearn.model_selection import train_test_split
```

```
xtrain, xtest, ytrain, ytest = train_test_split(x,y,train_size=0.8,random_state=6)
```

- Convert Categorical to Numerical Values if applicable.
- Check for Null Value.
- Divide the dataset into Independent(X) and Dependent(Y) variables.
- Split the dataset into training and testing datasets.
- Scale the Features if necessary.

4. Use Logistic regression(Train the Machine) to Create Model

```
from sklearn.linear_model import LogisticRegression
logic=LogisticRegression()
logic.fit(xtrain,ytrain)
```

5. Evaluate the model

```
ypred=logic.predict(xtest)
ypred
```

6. find the accuracy from sklearn.metrics

```
from sklearn.metrics import accuracy_score
score=accuracy_score(ytest,ypred)
score
```

7. Compute Confusion matrix to find TP, FP, TN, FN, Accuracy, Error rate, Precision, Recall

a. Print confusion matrix

```
from sklearn.metrics import confusion_matrix,classification_report
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

b. Print Classification Report

```
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

c. Find accuracy, precision, recall and f1_score using their formula.

```
accuracy=(TP+TN)/(TP+FP+TN+FN)
precision=TP/(TP+FP)
recall=TP/(TP+FN)
f1_Score=(2*precision*recall)/(precision+recall)
```

Conclusion: In this way we learned to implement Logistic Regression for classification type of problem and generated the classification report.

Lab Assignment 06

Title- Data Analytics III

Aim:

Implement naïve bayes classification on iris dataset and calculate confusion matrix

Problem Statement:

1. Implement Simple Naïve Bayes classification algorithm using Python/R on iris.csv dataset.
2. Compute Confusion matrix to find TP, FP, TN, FN, Accuracy, Error rate, Precision, Recall on the given dataset.

Objective:

Students should be able to do data analysis using Naïve Bayes classification algorithm using Python for any open source dataset

Prerequisite:

1. Basic of Python Programming
2. Concept of Baye's theorem.

Theory:

1. Concepts used in Naïve Bayes classifier
2. Confusion Matrix Evaluation Metrics

1. Concepts used in Naïve Bayes classifier

-
- REMEMBER: Union="or" ; Intersection="and"
 - $P(A \cup B) = P(A) + P(B) - P(A \cap B)$
 - $P(A) = P(A \cap B) + P(A \cap B')$
 - $P(A') = 1 - P(A)$
 - If $A \subset B$, then $P(A) \leq P(B)$

- Naïve Bayes algorithm is a supervised learning algorithm, which is based on **Bayes theorem** and used for solving classification problems.
- It is mainly used in *text classification* that includes a high-dimensional training dataset.

- Naïve Bayes Classifier is one of the simple and most effective Classification algorithms which helps in building the fast machine learning models that can make quick predictions.
- **It is a probabilistic classifier, which means it predicts on the basis of the probability of an object.**
- Some popular examples of Naïve Bayes Algorithm are **spam filtration, Sentimental analysis, and classifying articles.**
- **Naïve:** It is called Naïve because it assumes that the occurrence of a certain feature is independent of the occurrence of other features. Such as if the fruit is identified on the bases of color, shape, and taste, then red, spherical, and sweet fruit is recognized as an apple. Hence each feature individually contributes to identify that it is an apple without depending on each other.
- **Bayes:** It is called Bayes because it depends on the principle of **Bayes' Theorem.**

The formula for Bayes Theorem is

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Where,

P(A|B) is Posterior probability: Probability of hypothesis A on the observed event B.

P(B|A) is Likelihood probability: Probability of the evidence given that the probability of a hypothesis is true.

Conditional Probabilities:

The conditional probability can be calculated using the joint probability, although it would be intractable. Bayes Theorem provides a principled way for calculating the conditional probability. The simple form of the calculation for Bayes Theorem is as follows:

$$P(A|B) = P(B|A) * P(A) / P(B)$$

Algorithm

Step 1: Import libraries and create alias for Pandas, Numpy and Matplotlib

Step 2: Import the Iris dataset by calling URL.

```
iris_df = pd.read_csv(r"C:\Users\Student\Downloads\iris.csv")
```

Step 3: Initialize the data frame

Step 4: Perform Data Preprocessing

- Divide the dataset into Independent(X) and Dependent(Y) variables.
- Scale the Features if necessary.
- Convert Categorical to Numerical Values if applicable
- Check for Null Value

Step 5: Use Naive Bayes algorithm(Train the Machine) to Create Model

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score
```

Step 6: Separate features and target variable

```
X = iris_df[['Color', 'Height', 'Smelly']]
```

```
y = iris_df['species']
```

Step 7: Split the dataset into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Step 8: Initialize the Naïve Bayes classifier

```
nb_classifier = GaussianNB()
```

Train the classifier on the training data

```
nb_classifier.fit(X_train, y_train)
```

Predict the classes for the test set

```
y_pred = nb_classifier.predict(X_test)
```

Step 9: Compute the confusion matrix

```
conf_matrix = confusion_matrix(y_test, y_pred)
```

Calculate TP, FP, TN, FN

```
TP = conf_matrix[1, 1]
```

```
FP = conf_matrix[0, 1]
```

```
TN = conf_matrix[0, 0]
```

```
FN = conf_matrix[1, 0]
```

```
# Compute accuracy, error rate, precision, and recall
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
error_rate = 1 - accuracy
```

```
precision = precision_score(y_test, y_pred, average='weighted')
```

```
recall = recall_score(y_test, y_pred, average='weighted')
```

```
print("Confusion Matrix:")
```

```
print(conf_matrix)
```

```
print("\nTP:", TP)
```

```
print("FP:", FP)
```

```
print("TN:", TN)
```

```
print("FN:", FN)
```

```
print("\nAccuracy:", accuracy)
```

```
print("Error Rate:", error_rate)
```

```
print("Precision:", precision)
```

```
print("Recall:", recall)
```

Conclusion:

In this way we have done data analysis using Naive Bayes Algorithm for Iris dataset and evaluated the performance of the model.

Lab Assignment 7

Title: Text Analytics

PROBLEM STATEMENT:

1. Extract Sample document and apply following document preprocessing methods: Tokenization, POS Tagging, stop words removal, Stemming and Lemmatization.
2. Create representation of documents by calculating Term Frequency and Inverse Document Frequency.

THEORY:

Basic concepts of Text Analytics

- One of the most frequent types of day-to-day communication is text communication. In our everyday routine, we chat, message, tweet, share status, email, create blogs, and offer opinions and criticism. All of these actions lead to a substantial amount of unstructured text being produced. It is critical to examine huge amounts of data in this sector of the online world and social media to determine people's opinions.
- Text mining is also referred to as text analytics. Text mining is a process of exploring sizable textual data and finding patterns. Text Mining processes the text itself, while NLP processes with the underlying metadata. Finding frequency counts of words, length of the sentence, presence/absence of specific words is known as text mining. Natural language processing is one of the components of text mining. NLP helps identify sentiment, finding entities in the sentence, and category of blog/article. Text mining is preprocessed data for text analytics. In Text Analytics, statistical and machine learning algorithms are used to classify information.

Text Analysis Operations using natural language toolkit

NLTK(natural language toolkit) is a leading platform for building Python programs to work with human language data. It provides easy-to-use interfaces and lexical resources such as WordNet, along with a suite of text processing libraries for classification, tokenization, stemming, tagging, parsing, and semantic reasoning and many more. Analysing movie reviews is one of the classic examples to demonstrate a simple NLP Bag-of-words model, on movie reviews.

Tokenization:

- **Tokenization** is the first step in text analytics. The process of breaking down a text paragraph into smaller chunks such as words or sentences is called Tokenization. Token is a single entity that is the building blocks for a sentence or paragraph.
- **Sentence tokenization** : split a paragraph into list of sentences using `sent_tokenize()` method

Stop words removal:

- Stopwords considered as noise in the text. Text may contain stop words such as is, am, are, this, a, an, the, etc. In NLTK for removing stopwords, you need to create a list of stopwords and filter out your list of tokens from these words.

Stemming and Lemmatization

- **Stemming** is a normalization technique where lists of tokenized words are converted into shortened root words to remove redundancy. Stemming is the process of reducing inflected (or sometimes derived) words to their word stem, base or root form. A computer program that stems word may be called a stemmer. E.g. A stemmer reduces the words like fishing, fished, and fisher to the stem fish. The stem need not be a word, for example the Porter algorithm reduces, argue, argued, argues, arguing, and argus to the stem argu .
- **Lemmatization** in NLTK is the algorithmic process of finding the lemma of a word depending on its meaning and context. Lemmatization usually refers to the morphological analysis of words, which aims to remove inflectional endings. It helps in returning the base or dictionary form of a word known as the lemma. Eg. Lemma for studies is study

Lemmatization Vs Stemming

Stemming algorithm works by cutting the suffix from the word. In a broader sense cuts either the beginning or end of the word. On the contrary, Lemmatization is a more powerful operation, and it takes into consideration morphological analysis of the words. It returns the lemma which is the base form of all its inflectional forms. In-depth linguistic knowledge is required to create dictionaries and look for the proper form of the word. Stemming is a general operation while lemmatization is an intelligent operation where the proper form will be looked in the dictionary. Hence, lemmatization helps in forming better machine learning features.

POS Tagging

POS (Parts of Speech) tell us about grammatical information of words of the sentence by assigning specific token (Determiner, noun, adjective , adverb , verb,Personal Pronoun etc.) as tag (DT,NN ,JJ,VB,PRP etc) to each words. Word can have more than one POS depending upon the context where it is used. We can use POS tags as statistical NLP tasks. It distinguishes a sense of word which is very helpful in text realization and infer semantic information from text for sentiment analysis.

Text Analysis Model using TF-IDF

Term frequency–inverse document frequency(TFIDF) , is a numerical statistic that is intended to reflect how important a word is to a document in a collection or corpus.

- **Term Frequency (TF)** It is a measure of the frequency of a word (w) in a document (d). TF is defined as the ratio of a word's occurrence in a document to the total number of words in a document. The denominator term in the formula is to normalize since all the corpus documents are of different lengths.

$$TF(w, d) = \frac{\text{occurences of } w \text{ in document } d}{\text{total number of words in document } d}$$

Inverse Document Frequency (IDF)

It is the measure of the importance of a word. Term frequency (TF) does not consider the importance of words. Some words such as 'of', 'and', etc. can be most frequently present but are of little significance. IDF provides weightage to each word based on its frequency in the corpus D.

$$IDF(w, D) = \ln\left(\frac{\text{Total number of documents (N) in corpus } D}{\text{number of documents containing } w}\right)$$

Term Frequency — Inverse Document Frequency (TFIDF)

It is the product of TF and IDF. TFIDF gives more weight-age to the word that is rare in the corpus (all the documents). TFIDF provides more importance to the word that is more frequent in the document.

$$TFIDF(w, d, D) = TF(w, d) * IDF(w, D)$$

CONCLUSION:

We have performed Text Analysis experiment using TF-IDF algorithm

Lab Assignment 8

Title: Data Visualization I

PROBLEM STATEMENT:

1. Use the inbuilt dataset 'titanic'. The dataset contains 891 rows and contains information about the passengers who boarded the unfortunate Titanic ship. Use the Seaborn library to see if we can find any patterns in the data.
2. Write a code to check how the price of the ticket (column name: 'fare') for each passenger is distributed by plotting a histogram.

THEORY:

Data visualization is the graphical representation of information and data. By using visual elements like charts, graphs, and maps, data visualization tools provide an accessible way to see and understand trends, outliers, and patterns in data.

In the world of Big Data, data visualization tools and technologies are essential to analyze massive amounts of information and make data-driven decisions.

Common general types of data visualization:

Charts

Tables

Graphs

Maps

Infographics

Dashboards

More specific examples of methods to visualize data:

Area Chart

Bar Chart

Cartogram

Gantt Chart

Heat Map

Highlight Table

Histogram

Scatter Plot (2D or 3D)

Seaborn is a Python data visualization library based on matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics. Seaborn helps you explore and understand your data. Its plotting functions operate on dataframes and arrays containing whole datasets and internally perform the

necessary semantic mapping and statistical aggregation to produce informative plots. Its dataset-oriented, declarative API lets you focus on what the different elements of your plots mean, rather than on the details of how to draw them.

By convention, it is imported with the shorthand `sns`.

```
# Import seaborn
import seaborn as sns
```

Behind the scenes, seaborn uses matplotlib to draw its plots. For interactive work, it's recommended to use a Jupyter/IPython interface in matplotlib mode, or else you'll have to call `matplotlib.pyplot.show()` when you want to see the plot.

Now, let's perform the operations in the problem statement on our data set.

Loading the dataset and libraries -

In [1]:

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

dataset = sns.load_dataset('titanic')

dataset.head()
```

Out[1]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True

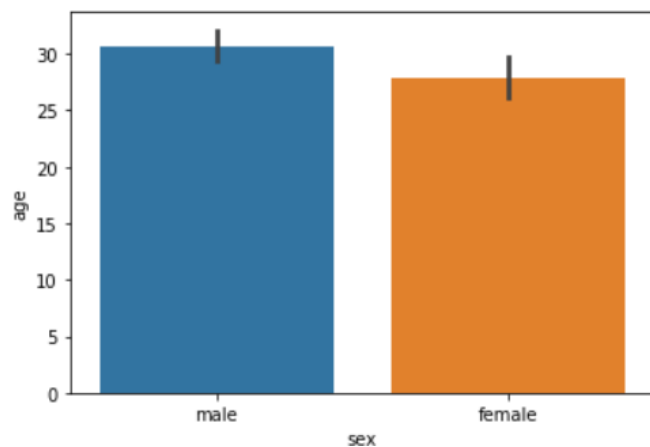
Some patterns can be seen by performing various operations like-

In [4]:

```
sns.barplot(x='sex', y='age', data=dataset)
```

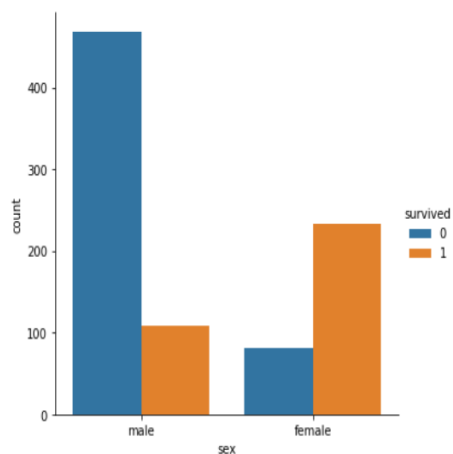
Out[4]:

<matplotlib.axes._subplots.AxesSubplot at 0x2329d50fe88>



In [9]: `sns.catplot(x="sex", hue="survived",
kind="count", data=dataset)`

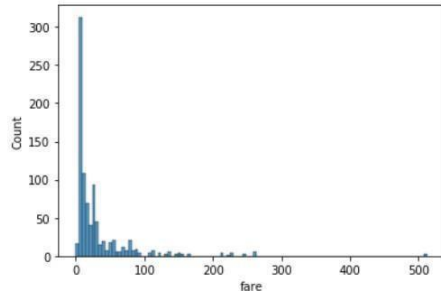
Out[9]: <seaborn.axisgrid.FacetGrid at 0x24be16f4448>



Assign a variable to x to plot a univariate distribution along the x axis:

```
In [5]: sns.histplot(data=dataset, x="fare")
```

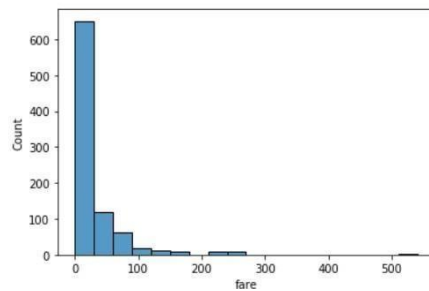
```
Out[5]: <matplotlib.axes._subplots.AxesSubplot at 0x16d988db688>
```



Check how well the histogram represents the data by specifying a different bin width:

```
In [8]: sns.histplot(data=dataset, x="fare", binwidth=30)
```

```
Out[8]: <matplotlib.axes._subplots.AxesSubplot at 0x16d9f3b3788>
```



CONCLUSION:

We have successfully implemented operations of the 'seaborn' library on the 'titanic' dataset, and explored some patterns in the data. We have also successfully plotted a histogram to see the ticket price distribution

Lab Assignment 9

Title: Data Visualization II

PROBLEM STATEMENT:

1. Use the inbuilt dataset 'titanic' as used in the above problem. Plot a box plot for distribution of age with respect to each gender along with the information about whether they survived or not. (Column names : 'sex' and 'age')
2. Write observations on the inference from the above statistics.

THEORY:

Data visualization is the graphical representation of information and data. By using visual elements like charts, graphs, and maps, data visualization tools provide an accessible way to see and understand trends, outliers, and patterns in data.

In the world of Big Data, data visualization tools and technologies are essential to analyze massive amounts of information and make data-driven decisions.

Common general types of data visualization:

Charts

Tables

Graphs

Maps

Infographics

Dashboards

More specific examples of methods to visualize data:

Area Chart

Bar Chart

Cartogram

Gantt Chart

Heat Map

Highlight Table

Histogram

Scatter Plot (2D or 3D)

Seaborn is a Python data visualization library based on matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics. Seaborn helps you explore and understand your data. Its plotting functions operate on dataframes and arrays containing whole datasets and internally perform the necessary semantic mapping and statistical aggregation to produce informative plots. Its dataset-oriented,

declarative API lets you focus on what the different elements of your plots mean, rather than on the details of how to draw them.

By convention, it is imported with the shorthand `sns`.

```
# Import seaborn
import seaborn as sns
```

Behind the scenes, seaborn uses matplotlib to draw its plots. For interactive work, it's recommended to use a Jupyter/IPython interface in matplotlib mode, or else you'll have to call `matplotlib.pyplot.show()` when you want to see the plot.

Now, let's perform the operations in the problem statement on our data set.

Loading the dataset and libraries -

In [1]:

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

dataset = sns.load_dataset('titanic')

dataset.head()
```

Out[1]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True

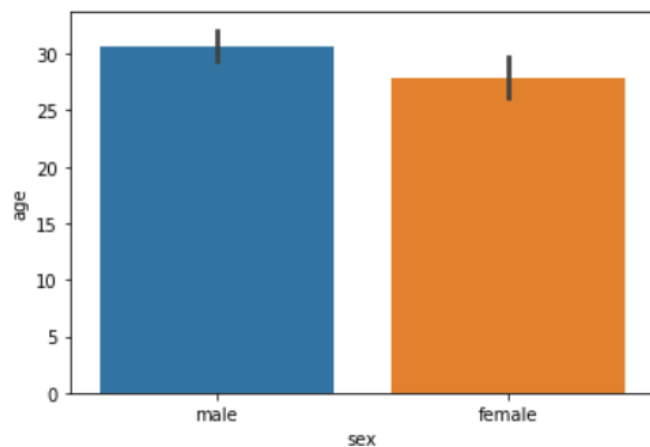
Some patterns can be seen by performing various operations like-

In [4]:

```
sns.barplot(x='sex', y='age', data=dataset)
```

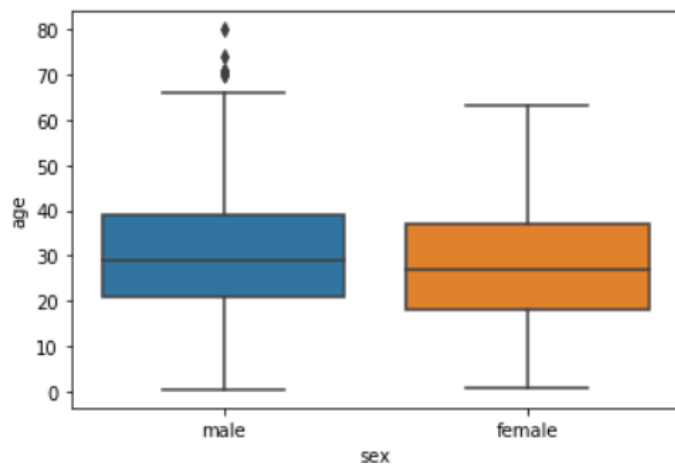
Out[4]:

<matplotlib.axes._subplots.AxesSubplot at 0x2329d50fe88>



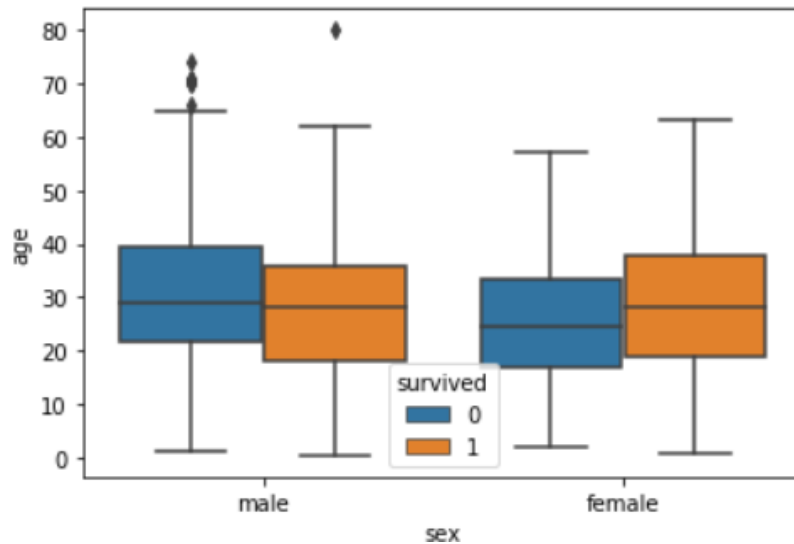
In [9]: `sns.boxplot(x='sex', y='age', data=dataset)`

Out[9]: <matplotlib.axes._subplots.AxesSubplot at 0x16d9f439988>



```
In [10]: sns.boxplot(x='sex', y='age', data=dataset, hue="survived")
```

```
Out[10]: <matplotlib.axes._subplots.AxesSubplot at 0x16d9f4bc948>
```



Inferences -

Let's try to understand the box plot for female. The first quartile starts at around 5 and ends at 22 which means that 25% of the passengers are aged between 5 and 25. The second quartile starts at around 23 and ends at around 32 which means that 25% of the passengers are aged between 23 and 32. Similarly, the third quartile starts and ends between 34 and 42, hence 25% passengers are aged within this range and finally the fourth or last quartile starts at 43 and ends around 65.

If there are any outliers or the passengers that do not belong to any of the quartiles, they are called outliers and are represented by dots on the box plot.

Now in addition to the information about the age of each gender, you can also see the distribution of the passengers who survived. For instance, you can see that among the male passengers, on average more younger people survived as compared to the older ones. Similarly, you can see that the variation among the age of female passengers who did not survive is much greater than the age of the surviving female passengers.

CONCLUSION:

We have successfully implemented operations of the 'seaborn' library on the 'titanic' dataset, and explored some patterns in the data. We have also successfully plotted a histogram to see the ticket price distribution

Lab Assignment 10

Title: Data Visualization III

PROBLEM STATEMENT:

Download the Iris flower dataset or any other dataset into a DataFrame (e.g., <https://archive.ics.uci.edu/ml/datasets/Iris>). Scan the dataset and give the inference as:

1. List down the features and their types (e.g., numeric, nominal) available in the dataset.
2. Create a histogram for each feature in the dataset to illustrate the feature distributions.
3. Create a boxplot for each feature in the dataset.
4. Compare distributions and identify outliers.

THEORY:

Histogram:-

Pandas.DataFrame.hist() function is useful in understanding the distribution of numeric variables. This function splits up the values into the numeric variables. Its main functionality is to make the Histogram of a given Data frame.

The distribution of data is represented by Histogram. When Function Pandas DataFrame.hist() is used, it automatically calls the function matplotlib.pyplot.hist() on each series in the DataFrame

Syntax: DataFrame.hist(data, column=None, by=None, grid=True, xlabelsize=None, xrot=None, ylabelsize=None, yrot=None, ax=None, sharex=False, sharey=False, figsize=None, layout=None, bins=10, backend=None, legend=False, **kwargs)

Parameters:

data: DataFrame

column: str or sequence

xlabelsize: int, default None

ylabelsize: int, default None

ax: Matplotlib axes object, default None

**kwargs

All other plotting keyword arguments to be passed to matplotlib.pyplot.hist().

Return:

matplotlib.AxesSubplot or numpy.ndarray

Box Plot :-

Box Plot is the visual representation of the depicting groups of numerical data through their quartiles. Boxplot is also used for detect the outlier in data set. It captures the summary of the data efficiently with a simple box and whiskers and allows us to compare easily across groups. Boxplot summarizes a sample data using 25th, 50th and 75th percentiles. These percentiles are also known as the lower quartile, median and upper quartile.

A box plot consist of 5 things.

- Minimum
- First Quartile or 25%
- Median (Second Quartile) or 50%
- Third Quartile or 75%
- Maximum

Draw the boxplot using seaborn library:

Syntax :
`seaborn.boxplot(x=None, y=None, hue=None, data=None, order=None, hue_order=None, orient=None, color=None, palette=None, saturation=0.75, width=0.8, dodge=True, fliersize=5, linewidth=None, whis=1.5, notch=False, ax=None, **kwargs)`

Parameters:

x = feature of dataset

y = feature of dataset

hue = feature of dataset

data = dataframe or full dataset

color = color name

Identify outliers:-

Detect and Remove the Outliers using Python

An Outlier is a data-item/object that deviates significantly from the rest of the (so-called normal)objects. They can be caused by measurement or execution errors. The analysis for outlier detection is referred to as outlier mining. There are many ways to detect the outliers, and the removal process is the data frame same as removing a data item from the panda's data frame.

Here pandas data frame is used for a more realistic approach as in real-world project need to detect the outliers arouse during the data analysis step, the same approach can be used on lists and series-type objects.

Detecting the outliers

Outliers can be detected using visualization, implementing mathematical formulas on the dataset, or using the statistical approach. All of these are discussed below.

1. Visualization

- **Using Box Plot**
- **Using ScatterPlot**
- **Z-score**

Z- Score is also called a standard score. This value/score helps to understand that how far is the data point from the mean. And after setting up a threshold value one can utilize z score values of data points to define the outliers.

$$Zscore = (data_point - mean) / std. deviation$$

- **IQR (Inter Quartile Range)**

Inter Quartile Range approach to finding the outliers is the most commonly used and most trusted approach used in the research field.

$$IQR = Quartile3 - Quartile1$$

Procedure & Code:-

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [2]: data1 = pd.read_csv("C:/Users/Admin/Downloads/Iris.csv")
data1.head()
```

```
Out[2]:
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

```
In [3]: print(data1.columns)

Index(['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm',
      'Species'],
      dtype='object')
```

```
In [7]: data1.info()

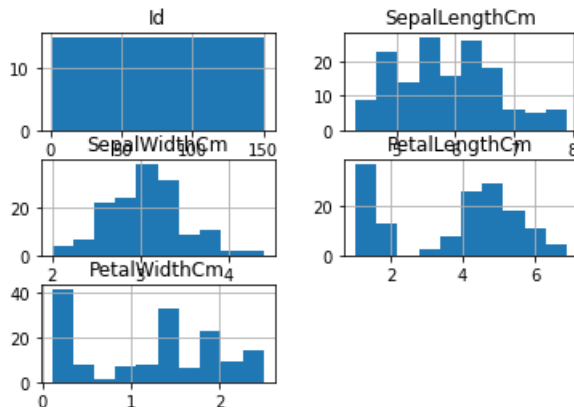
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   Id                    150 non-null   int64  
1   SepalLengthCm         150 non-null   float64
2   SepalWidthCm          150 non-null   float64
3   PetalLengthCm         150 non-null   float64
4   PetalWidthCm          150 non-null   float64
5   Species               150 non-null   object  
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

```
In [13]: data1.dtypes
```

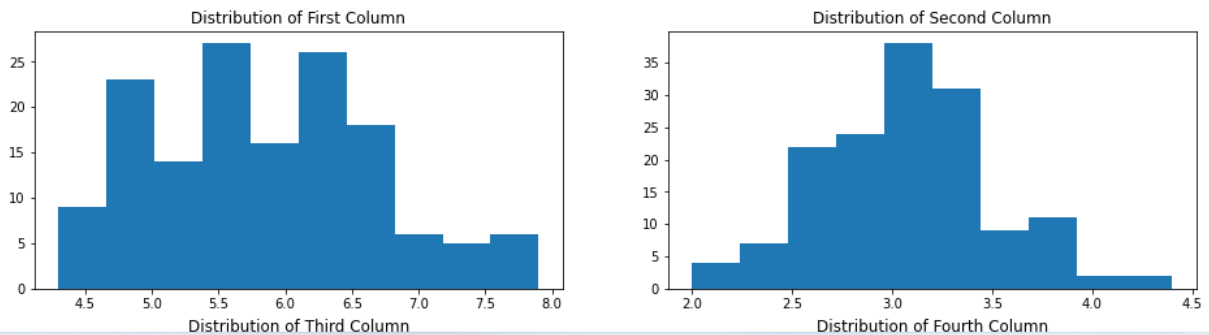
```
Out[13]: Id                    int64
SepalLengthCm                float64
SepalWidthCm                  float64
PetalLengthCm                 float64
PetalWidthCm                  float64
Species                       object
dtype: object
```

```
In [20]: data1.hist()
```

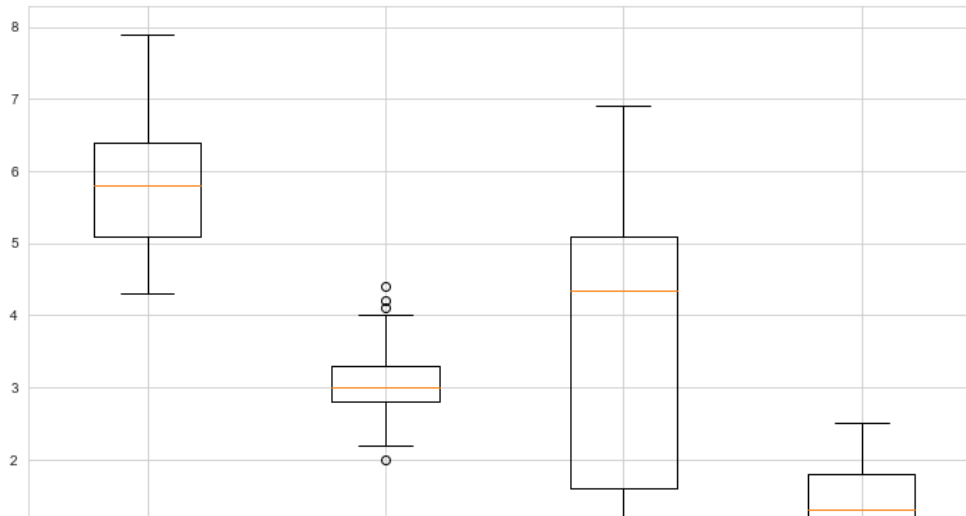
```
Out[20]: array([[<AxesSubplot:title={'center':'Id'}>,  
                <AxesSubplot:title={'center':'SepalLengthCm'}>],  
               [<AxesSubplot:title={'center':'SepalWidthCm'}>,  
                <AxesSubplot:title={'center':'PetalLengthCm'}>],  
               [<AxesSubplot:title={'center':'PetalWidthCm'}>],  
               dtype=object)
```



```
In [17]: fig, axes = plt.subplots(2, 2, figsize=(16, 8))  
  
axes[0,0].set_title("Distribution of First Column")  
axes[0,0].hist(data1["SepalLengthCm"]);  
  
axes[0,1].set_title("Distribution of Second Column")  
axes[0,1].hist(data1["SepalWidthCm"]);  
  
axes[1,0].set_title("Distribution of Third Column")  
axes[1,0].hist(data1["PetalLengthCm"]);  
  
axes[1,1].set_title("Distribution of Fourth Column")  
axes[1,1].hist(data1["PetalWidthCm"]);
```



```
In [39]: data_to_plot = [data1["SepalLengthCm"],data1["SepalWidthCm"],data1["PetalLengthCm"],data1["PetalWidthCm"]]
# Creating a figure instance
fig = plt.figure(1, figsize=(12,8))
# Creating an axes instance
ax = fig.add_subplot(111)
# Creating the boxplot
bp = ax.boxplot(data_to_plot);
```

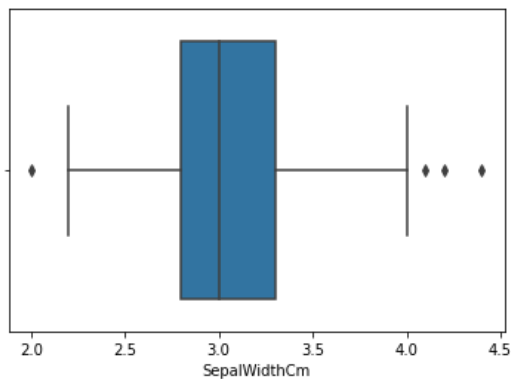


```
In [19]: sns.boxplot(data1['SepalWidthCm'])
```

C:\ProgramData\Anaconda3\lib\site-packages\seaborn_decorators.py:36: FutureWarning: Pass the following variable as ng: x. From version 0.12, the only valid positional argument will be `data`, and passing other arguments without keyword will result in an error or misinterpretation.

warnings.warn()

```
Out[19]: <AxesSubplot:xlabel='SepalWidthCm'>
```



```
In [36]: print(np.where(data1['SepalWidthCm']>4.0))
(array([15, 32, 33], dtype=int64),)
```

CONCLUSION:

We have successfully implemented operations on the 'iris' dataset, also we have successfully plotted a histogram, boxplot and identified outliers.